Member 4 Briefing: Validation, Results, Build, and Delivery

Beginner foundation: this guide uses no assumed software background. Read this section before the test lists and numerical results.

Beginner foundation: how we know a program is trustworthy

Writing code is not the same as proving it works. We need validation: deliberate evidence that the program behaves correctly on stated cases.

Build, run, and test are different actions

Word - Plain meaning

Source code - Human-written program instructions, such as .cpp files.

Compiler - Software that turns C++ source code into a runnable program.

Build - The complete process of creating runnable programs from source code.

Run - Start a runnable program with input.

Test - A deliberate check with an expected result.

Result - The output/evidence produced by a run or test.

What are CMake and CTest?

CMake helps organize a C++ build. It reads CMakeLists.txt, which lists what source files belong in each program and test. CMake prepares instructions for the compiler. CTest is the accompanying test runner: it runs the tests that the build registered. Neither CMake nor CTest changes the mathematical algorithm; they help make the project repeatable.

What numerical evidence means

The program calculates an approximate x for A*x=b. We measure the residual, b-A*x, to see what remains wrong. We also measure backward error, which asks how much the original problem would need to change for this computed x to become exact. Small backward error is evidence that the result fits the original equation well.

The key rule for presentation

Tests can support a claim about the cases that were tested. They do not support a claim that every possible input works, that the GPU is always faster, or that a method not in the source code was implemented. The rest of the guide gives the exact recorded evidence and safe wording.

Validation words decoded before they appear

Word - Beginner meaning

Regression test - A repeatable test kept to catch an old bug returning later.

Test fixture - A small controlled input used by a test.

Snapshot - A saved record of expected output values for comparison.

Decimal80 - A higher-precision decimal calculation used by checkers as an independent reference.

CLI - Command-line interface: run a program by typing its name and arguments.

Exit code - A small number returned by a program; 0 conventionally means success.

Manifest - A list of files and integrity information in a package.

Archive - A compressed package that preserves older files together.

A practical reading route

Read Sections 1 and 3 first. Then learn the safe/unsafe claims in Section 6. Build commands and archive details are useful reference material; do not try to memorize them unless you are demonstrating the project live.

Your role in one sentence

Explain how the project checks numerical correctness, what evidence is current, what is archived historical evidence, and how to present results responsibly.

Core message

The project has strong regression and small-fixture correctness evidence for the active CPU and CUDA solvers. It does not currently make a fresh performance benchmark claim. Historical campaigns and raw results are preserved in an archive and must be described as historical unless rerun under a recorded, comparable methodology.

1. What is being validated?

The solver aims to solve Ax=b, so validation should examine the original system, not only whether a program exits successfully.

Residual

$$r = b - Ax.$$

A small residual means the computed solution nearly satisfies the equation.

Normwise backward error

$$\eta(x) = \frac{\|b - Ax\|_\infty}{\|A\|_\infty \|x\|_\infty + \|b\|_\infty}.$$

This asks: how much would the original data need to change for the computed x to be an exact solution? It is the outer solver's final acceptance criterion.

Inner GMRES residual

GMRES separately uses:

$$\|Pr - PAd\|_2 / \|Pr\|_2.$$

That checks the correction equation, while backward error checks the original system after the correction is

added. Do not confuse them.

2. Active test layout

Test file/group - What it checks

tests/test_cpu.cpp - CSR construction, Matrix Market parsing, basic CPU matrix behavior.

tests/test_spai.cpp - FP64 SPAI construction and numerical edge cases.

tests/test_gmres.cpp - FP64 GMRES behavior and failures.

tests/test_ir.cpp - FP64 end-to-end refinement behavior.

tests/test_fp32.cpp - FP32 CPU behavior.

tests/test_cuda_*.cpp - CUDA FP64/FP32 sparse operations, SPAI, GMRES, and IR.

tests/release_cpu_snapshot.cpp - Produces fixed numerical snapshots.

tests/check_release_cpu_decimal.py - Independently checks CPU snapshots using Decimal80-style arithmetic.

tests/check_cuda_*_decimal.py - Independent CUDA numerical reconstruction/checking.

tests/check_solver_cli.py - Command-line behavior for all four solver programs.

tests/test_submission_tools.py - Packaging/archive safety behavior.

The Python checkers are deliberately independent of the solver's internal return values: their role is to recompute relevant quantities rather than simply trust the program's own report.

3. Current recorded results

The final cleanup record reports:

- CPU CTest: 5/5 suites pass.
- CUDA CTest: 8/8 suites pass.
- CLI checks: 32/32 pass across the four solver programs.
- CPU fixed-snapshot Decimal80 check: 16 records and 500 checks pass.
- CUDA independent Decimal80 checks: zero recorded failures across the listed

SPAI, GMRES, and IR snapshots.

The recorded small-fixture backward errors are approximately:

Backend - FP32 - FP64

CPU - 4.34993e-8 - 5.40158e-17

GPU - 1.44998e-8 - 2.02559e-17

These are correctness checks on deterministic small fixtures, not timings or a claim about every matrix. State the scope every time you cite them.

4. Build and test commands

For a GCC/Clang CPU build:

```
cmake -S . -B build/cpu -DCMAKE_BUILD_TYPE=Debug
cmake --build build/cpu
ctest --test-dir build/cpu --output-on-failure
```

For the Windows helper:

```
scriptsbuild_windows.cmd cpu
scriptsbuild_windows.cmd cuda 89
```

For a portable CUDA configuration, assuming the necessary CUDA environment:

```
cmake -S cuda -B build/cuda -DCMAKE_CUDA_ARCHITECTURES=89 -DCMAKE_BUILD_TYPE=Debug
cmake --build build/cuda
ctest --test-dir build/cuda --output-on-failure
```

The test environment matters. The recorded final environment was Windows x64, GCC 6.3 for the CPU reference build, MSVC 19.44, CUDA 13.4, and an RTX 4060 Laptop GPU. Compiler, driver, GPU, flags, matrix,

tolerances, and stopping rules all affect reproducibility.

5. Understanding the archive

archives/research_20261019.zip preserves historical matrices, raw traces, campaign tools, profiles, reports, and provenance. The loose historical files were archived to reduce working-tree size; they were not discarded.

Use:

```
python scripts/research_archive.py verify
python scripts/research_archive.py restore
```

The archive is about 633 MB compressed and expands to about 1.85 GB. The active source submission deliberately excludes it. Do not restore it casually just to present the project; it is needed only when inspecting or reproducing older campaign evidence.

6. How to phrase claims safely

Good claims

- "The final recorded regression suites passed 5/5 CPU and 8/8 CUDA tests."
- "The solver validates final answers through original-system backward error."
- "Independent Decimal80-oriented checks were used for recorded numerical snapshots."
- "The archived experiments preserve earlier results and provenance."
- "The implementation includes uniform FP32 and FP64 CPU/GPU variants."

Claims to avoid

- "We proved the solver converges for all sparse matrices."
- "GPU is faster" without a specific controlled benchmark table.
- "The project implements BSPAI/mixed precision."
- "CPU and GPU are bitwise identical."
- "All historical archived results were freshly rerun."

7. Presentation-ready results slide

```
Validation summary

* CPU regression suite: 5/5 passed
* CUDA regression suite: 8/8 passed
* Solver CLI/error-path checks: 32/32 passed
* Independent fixed-snapshot numerical checks: passed
* Small-fixture backward error: about 10^-8 in FP32; about 10^-17 in FP64

Scope: deterministic correctness validation; not a fresh performance claim.
```

8. Suggested speaking script (about 70 seconds)

"We validate more than whether the executable runs. For each solution, the important original-system quantity is the normwise backward error, which measures how closely the computed answer satisfies Ax=b. GMRES also checks its preconditioned correction residual, but final acceptance uses the original system. The recorded final state has five out of five CPU regression suites, eight out of eight CUDA suites, and thirty-two out of thirty-two command-line checks passing. Independent Decimal80-oriented checkers also passed on the recorded snapshots. These are correctness results on controlled fixtures, not a claim of a general benchmark speedup. Older experimental artifacts remain preserved in a reproducible archive."

Likely questions and safe answers

9. A validation ladder: what every layer proves

A single successful run is weak evidence. This project uses layers, each designed to catch a different class of mistake.

- 1. Build validation proves that the selected compiler, headers, and libraries can turn the source into executables. It does not prove numerical correctness.
- 2. Unit or focused tests exercise a small component such as parsing, CSR multiplication, a norm, or a stopping branch.
- 3. Algorithm tests exercise SPAI, GMRES, and iterative refinement on known fixtures, including failure paths.
- 4. Cross-variant checks compare CPU/GPU and FP32/FP64 behavior without incorrectly requiring every rounded intermediate value to be identical.
- 5. High-precision reference checks use Decimal80 calculations as an independent yardstick for residuals and solutions.
- 6. Sanitizer or allocation checks seek memory misuse and unintended dense storage.
- 7. Performance experiments measure only after numerical output, statuses, and iteration behavior are shown to remain acceptable.
- 8. Package validation proves that the delivered archive contains the intended source, documents, and reproducibility information-not merely that the developer's working tree passed.

10. What is measured mathematically?

Raw residual

$$r = b - Ax.$$

If r is zero, x exactly satisfies the stored arithmetic equation. A small raw residual is useful, but its meaning depends on the scale of A, b, and x.

Whole-solution normwise backward error

$$\eta = \frac{\|b - Ax\|_\infty}{\|A\|_\infty \|x\|_\infty + \|b\|_\infty}.$$

This asks: how small a scale-relative perturbation would make the computed x an exact solution? The infinity norm of a vector is its largest absolute entry; for a matrix it is the largest absolute row sum. Current bounds are based on 10*2^-53 for FP64 and 10*2^-24 for FP32.

GMRES correction verification

$$\rho = \frac{\|Pr - PAd\|_2}{\|Pr\|_2}.$$

Here d is the proposed correction. This ratio is recomputed using the real sparse operations after the GMRES recurrence. Acceptance limits are 1e-8 in FP64 and 1e-4 in FP32. This prevents an optimistic internal estimate from being mistaken for a verified correction.

Forward error versus backward error

Forward error compares x with a known x_true. It is meaningful on generated fixtures where the true solution is known. Backward error does not require the true solution and is generally more appropriate for arbitrary input matrices. A poorly conditioned matrix can have a small backward error but a larger forward error, because small data changes can naturally cause a large solution change.

11. Recorded correctness results

The active recorded baseline is:

- CPU CTest: 5/5 passed.
- CUDA CTest: 8/8 passed.
- Four solver command-line variants: 32 Decimal80 checks passed.
- CPU snapshots: 16 records and 500 Decimal80 checks.

On the deterministic small fixture, recorded final whole-solution errors were approximately:

Variant - Recorded error

CPU FP32 - 4.34993e-8

CPU FP64 - 5.40158e-17

GPU FP32 - 1.44998e-8

GPU FP64 - 2.02559e-17

The FP64 values are much smaller because FP64 carries 53 significand bits rather than FP32's 24. The GPU and CPU numbers need not be bit-for-bit equal: parallel reduction order, tied SPAI candidates, and library implementations can change rounding while both results satisfy the numerical contract.

These counts are only meaningful with context. CTest reports registered automated tests. Decimal80 checks compare selected outputs against a high-precision decimal reference. Neither count means the solver has been proven correct for every possible sparse matrix.

12. Performance results, discussed rather than merely listed

Persistent FP32 CUDA GMRES context

With a fixed preconditioner and isolated refinement, baseline medians were 10.50-10.85 ms and context-reuse medians were 9.71-9.80 ms, approximately 7-10% lower. The reason is straightforward: invariant A, P, handles, descriptors, work vectors, and workspaces no longer have to be rebuilt for each correction.

However, complete GPU SPAI-GMRES-IR time remained around 2.15-2.16 seconds with no clear overall change. SPAI construction is much larger than the saved setup cost on this case. The honest conclusion is

"context reuse improves the isolated repeated-correction path," not "the entire solver is 10% faster."

Lazy FP32 GMRES storage

At n=12000, eager allocation could request 576,048,000 bytes for the basis before knowing how many iterations were needed. The slab design begins a one-step solve with 768,000 bytes of device basis and 1,088 bytes of host projected-system storage; an exact initial solution requests neither.

The identity one-step runtime changed from 220.103 ms to 1.331 ms, and the exact-initial runtime from 16.634 ms to 0.907 ms. These dramatic ratios mainly expose avoided allocation/setup waste in deliberately easy cases; they should not be generalized to hard systems that actually require many basis vectors.

For the more representative recorded stencil_8192 FP32 workload, pooled medians were 2639.473 ms before and 2290.805 ms after: 348.668 ms lower, 13.21% less time, or about 1.15x speedup. The comparison retained matching status, construction counts, two outer refinement steps, fifteen total inner iterations, backward error, and serialized FP32 solution. Those matching numerical observations are what make the timing comparison credible.

Fused GPU SPAI residual work

The retained fused path reuses the fitted residual for candidate scoring and diagnostics. An exploratory stencil constructor median was approximately 2153 ms compared with 2282-2307 ms before, roughly 5-7% lower. A different row-parallel proposal was measured at 5-7% slower and was rejected. This demonstrates measurement-driven engineering, but still only on a limited workload and device.

13. Reproducibility checklist for every result

Before placing a number on a slide, record:

- exact matrix name, dimensions, nonzero count, and source;
- CPU/GPU model and available memory;
- operating system, compiler, CUDA toolkit, and library versions;
- executable and precision variant;
- SPAI tolerance, maximum growth steps, and candidate policy;
- GMRES tolerance and iteration cap;
- IR tolerance and step cap;
- random seed, if any randomness exists;
- warm-up policy, number of repetitions, and statistic reported;
- whether the time includes file input, SPAI construction, transfers, and verification;
- status, iteration counts, backward error, and solution comparison;
- exact command and revision.

A median is the middle observation after sorting repetitions. It is often more stable than a mean when occasional startup or system interruptions create outliers. A pooled median combines observations from the specified runs; it must not be confused with a result across many machines.

14. Delivery: source tree versus archive

The working directory is the development source of truth. A ZIP is a frozen copy prepared for delivery. Passing tests in the working directory does not guarantee that the ZIP contains the same files.

The current submission/SPAI_GMRES_IR_Submission.zip is known to differ from the recorded final validated package. The current archive is 147,400 bytes with 64 members; the recorded validated package was 313,131 bytes with 99 files. Therefore the current archive must not be described as the final verified submission.

A safe release process is:

- 1. Start from the exact intended revision and confirm the working tree contents.
- 2. Perform clean CPU and CUDA configuration/builds in documented environments.
- 3. Run CTest and the independent numerical checks.

- 4. Generate all required reports and guides.
- 5. Create a new archive from an explicit inclusion list.
- 6. List archive members and verify required source, headers, CMake files, docs, and test assets.
- 7. Extract into a fresh temporary directory.
- 8. Rebuild and retest the extracted copy.
- 9. Record archive size, member count, and a cryptographic hash.
- 10. Obtain independent verification before labeling it final.

A cryptographic hash is a short fingerprint computed from every byte. If one byte changes, the fingerprint is overwhelmingly likely to change. It identifies an exact artifact; it does not itself prove that the artifact is good.

15. Status table for the presentation

Claim - Evidence status - Safe wording

Four uniform-precision variants exist - Implemented and tested - "CPU/GPU FP32/FP64 SPAI-GMRES-IR variants are present."

Current registered tests pass - Recorded 5/5 CPU, 8/8 CUDA - Include environment and date in detailed report.

High-precision checks pass - Recorded CLI and CPU snapshot checks - Explain what Decimal80 checks compare.

Lazy storage helps - Supported on recorded cases - Quote the workload and matching outputs.

Persistent context helps end-to-end - Not clearly shown - Say isolated refinement improved; whole solver was unchanged within observation.

GPU is always faster - Unsupported - Do not claim this.

BSPAI is complete - Unsupported - Active implementation is SPAI.

Mixed precision is complete - Unsupported - Active variants use uniform precision.

Current ZIP is final verified delivery - False at present - Regenerate and independently verify first.

16. A five-minute validation narrative

Begin with the numerical contract: each returned solution is independently checked using the original equation and a scale-aware backward error. Then explain the correction-level GMRES verification. Next show the test layers and counts, followed by the small-fixture FP32/FP64 results. Present performance only after showing matching status, counts, iterations, and solution evidence. End with limitations and the archive discrepancy. This order makes it clear that speed was never accepted at the expense of correctness or reproducibility.

17. Validation flow from executable to evidence

- 1. solver_main.cpp::main returns a nonzero exit status if parsing, construction, GMRES, IR, or final acceptance fails.
- 2. SpaiResult/Fp32SpaiResult provides per-column patterns, fit histories, QR information, residuals, statuses, and assembled sparse matrices.
- 3. GmresResult/Fp32GmresResult provides estimated and explicitly recomputed residual evidence per iteration plus a final status.
- 4. IrResult provides the final solution, every outer diagnostic, every inner result, correction RHS vectors, and flags showing whether updates were applied.
- 5. CudaGmresResult adds transfer counts. CudaIrResult adds context construction/matrix upload counts, per-inner transfer deltas, and GPU SPAI telemetry.
- 6. Focused tests independently recompute sparse residuals, compare status contracts, inspect allocations, verify canonical structure, and emit oracle records.
- 7. Decimal80 scripts read the serialized outputs and compare them with wider independent arithmetic.
- 8. CTest aggregates registered executables and reports pass/fail at the build level.

18. CUDA timing and diagnostic fields

CudaSpaiTelemetry is designed to explain where GPU construction work went, not merely produce one elapsed time.

Field group - What it diagnoses

device_allocation_calls, frees, reallocations - allocation churn

allocated/freed/current/peak bytes - memory lifetime and peak pressure

max_single_allocation_bytes - largest single risk to device memory

sparse_b_bytes, packed_output_bytes - proof that main storage follows sparse nonzeros rather than n^2

reduced rows/columns/elements - actual QR problem sizes

QR workspace and slot bytes - vendor temporary-memory demand

nnz_b, nnz_n, nnz_p - sparse representation sizes

upload/download counts and bytes - PCIe/host-device traffic

transfer size buckets - whether traffic is dominated by tiny control messages or bulk arrays

synchronization counters - places where concurrency is forced to wait

streams/batch width/QR submissions - exposed column-level concurrency

GPU score evaluations - amount of scoring delegated to GPU

setup/adaptive-fit/packing wall times - coarse phase localization

The wall-time fields use host clocks. They include host waiting and scheduling effects and are not CUDA event measurements. They are diagnostic-only and never alter numerical decisions.

CudaGmresTransfers separately reports vector uploads/downloads, Arnoldi vector transfers, scalar observations, dot/norm/finite-flag observations, and scalar inputs used by AXPY, SpMV, and division. The desired signature is persistent matrix uploads plus small scalar observations-not repeated whole-matrix copies.

19. How the optimization comparisons were made credible

For lazy FP32 storage on stencil_8192, the comparison checked more than elapsed time: status, construction counts, two outer steps, fifteen inner iterations, backward error, and serialized FP32 solution all matched. Only after those checks was the median comparison-2639.473 ms versus 2290.805 ms-reported.

For persistent context, the isolated fixed-P path was measured separately from the complete solver. This exposed a 7-10% local saving while also showing why complete time around 2.15-2.16 s did not visibly change: SPAI construction dominated.

For the fused SPAI residual, the retained implementation was compared against recorded constructor timing and full tests. The alternative row-parallel implementation was slower and removed. Reporting a rejected optimization protects the work from hindsight bias and shows the professor that design choices were measurement-driven.

20. Test files and the questions they answer

- test_cpu.cpp: does Matrix Market/CSR behavior match its contract?

- test_spai.cpp: do patterns, coefficients, statuses, residuals, and scoring agree with independent checks?

- test_gmres.cpp: do estimated and recomputed residuals, easy exits, failures, and solutions behave correctly?

- test_ir.cpp: are outer diagnostics, update history, stopping, and failure propagation correct?

- test_cuda_spmv.cpp / test_fp32_cuda_spmv.cpp: does cuSPARSE CSR multiplication agree with CPU/reference values?

- test_cuda_gmres.cpp / test_fp32_cuda_gmres.cpp: does GPU Arnoldi/verification obey the numerical contract and transfer policy?

- test_cuda_ir.cpp: does CPU-staged outer refinement preserve inner results and transfer accounting?

- test_cuda_spai.cpp / test_fp32_cuda_spai.cpp: does native GPU construction agree within precision-appropriate bounds?

- test_cuda_spai_sparse.cpp: are sparse topology, independent full residuals, slot capacities, packed output,

and memory telemetry consistent?

- cuda_spai_allocation_audit.cu: intercepts source allocation/free calls so telemetry is checked against actual device allocations.

- release_cpu_snapshot.cpp: serializes a broad CPU behavior snapshot for regression and Decimal80 checking.

21. Result statements suitable for a parallel-computing professor

The CUDA design exposes two types of parallelism: data parallelism inside SpMV, vector operations, reductions, residual/scoring kernels, and QR; and task concurrency across independent SPAI column-fit streams. Serial dependencies remain in adaptive pattern growth, candidate selection, outer IR decisions, and the small Hessenberg/Givens calculation. Therefore performance is limited by both GPU arithmetic and host-device synchronization/control dependencies.

The current evidence supports memory-footprint and localized timing improvements. It does not yet include a broad strong-scaling study, GPU occupancy/bandwidth roofline analysis, CUDA-event kernel breakdown, Nsight profile, multi-GPU scaling, or a matrix-suite geometric mean. Those would be appropriate next experiments before making a general parallel-performance claim.

What does 5/5 or 8/8 mean? It is the number of registered CPU or CUDA regression suites that passed in the final recorded build.

Why use backward error rather than only forward error? The exact solution is often unknown, while backward error directly tests how well the computed solution satisfies the original problem.

Why is FP32 error larger than FP64 error? Float has much lower precision, so its attainable accuracy is correspondingly lower.

Where are the benchmark results? Historical data is preserved in the research archive. The active documented result is a correctness-validation record, not a newly rerun benchmark campaign.

What should be handed in? submission/SPAI_GMRES_IR_Submission.zip; it is the compact source submission with code, tests, specifications, and manifest.